

Viatom Product Communicate Protocol

Viatom Product Communicate Protocol

1. Overview
2. Protocol package structure
 - 2.1 Head
 - 2.2 CMD
 - 2.3 ~CMD
 - 2.4 Pkg. Type
 - 2.5 Pkg. No.
 - 2.6 Length
 - 2.7 Data
 - 2.8 CRC
3. Command
 - 3.1 Universal command(0xE0-0xFF)
 - 3.1.1 0xE0: Echo
 - 3.1.2 0xE1: Get device info
 - 3.1.3 0xE2: Reset
 - 3.1.4 0xE3: Factory Reset
 - 3.1.5 0xE4: Battery
 - 3.1.6 0xF1: Get file list
 - 3.1.7 0xF2: Read file start
 - 3.1.8 0xF3: Read file content
 - 3.1.9 0xF4: Read file end
 - 3.1.10 0xEC: Set time
 - 3.2 Private command(0x00 - 0xDF)
 - 3.2.1 Blood Pressure Monitor (BP2/BP2A)
 - 3.2.1.3 0x08: Get real-time data
 - 3.2.1.4 0x09: Switch device status
 - 3.2.2 ECG recorder (ER2)
 - 3.2.2.1 0x01: Get real-time waveform
 - 3.2.2.2 0x02: Get device run state
 - 3.2.2.3 0x03: Get real-time data
 - 3.2.3 Visualbeat(ER1)
 - 3.2.3.1 0x01: Get real-time waveform
 - 3.2.3.2 0x02: Get device run state
 - 3.2.3.3 0x03: Get real-time data

1. Overview

This Communicate protocol supports Bluetooth communicate, RS232 serial communication, USB HID .etc. the related Bluetooth service & characteristic UUID, USB PID/VID information and written in the `private command`.

Notes:

1. All the numerical values are in little endian.

2. As for USB HID communicate, every frame (64 bytes) data format is:

-	length	valid content	invalid content
Size	1 byte	length	63 - length
Description	valid content length	content	0x00

2. Protocol package structure

	Head	CMD	~CMD	Pkg. Type	Pkg No.	Length	Data	CRC
Size	1	1	1	1	1	2	Length	1
Description	0xA5	command	~command		0x00 - 0xFE	Data Lenght		CRC8

2.1 Head

Fixed value: 0xA5

2.2 CMD

refer to chapter 3: Command

2.3 ~CMD

inverse of CMD

2.4 Pkg. Type

Pkg. Type	Description
Regular Package (0x00-0x0F)	
0x00	Regular Request
0x01	Regular Response
...	
Irregular Private Package(0xA0-0xDF)	
...	refer to <code>Private Command</code>
...	
Irregular Universal Package(0xE0-0xFF)	
0xE0	unable to find file
0xE1	read file failed
0xE2	write file failed
0xE3	firmware update failed
0xE4	language pack update failed
...	
0xFB	device busy
0xFC	illegal package
0xFD	package command not support
0xFE	...
0xFF	regular error

2.5 Pkg. No.

package serial number, self-increasing, 0x00 - 0xFE. (0xFF is reserved)

2.6 Length

Length of Data, the value is 0 when there is no Data

2.7 Data

Data

2.8 CRC

CRC8 checksum for the package, the function is:

```
1  //*****
2  // function: crc8_compute
3  // input: pdata:data pointer, data_size:data size, crc_in:last crc value, default
   // is 0.
4  // output: crc8
5  //*****
6  unsigned char crc8_compute(unsigned char *pdata, unsigned data_size, unsigned char
   crc_in)
7  {
8      uint8_t cnt;
9      uint8_t crc_poly = 0x07;
10     uint8_t data_tmp = 0;
11
12     while (data_size--){
13         data_tmp = *(pdata++);
14         crc_in ^= (data_tmp << 0);
15         for(cnt = 0; cnt < 8; cnt++){
16             if(crc_in & 0x80){
17                 crc_in = (crc_in << 1) ^ crc_poly;
18             }else{
19                 crc_in = crc_in << 1;
20             }
21         }
22     }
23
24     return crc_in;
25 }
```

3. Command

Send: Package send from master device to slave device, Pkg. Type = 0x00

Receive: Package response from slave device to master device, Pkg. Type = 0x01

3.1 Universal command(0xE0-0xFF)

3.1.1 0xE0: Echo

Send		Description
Length	0XXXXX	data length
Data	0XXXXX	any
Receive		Description
Length	0XXXXX	data length
Data	0XXXXX	same as the Send Data

3.1.2 0xE1: Get device info

Send		Description
Length	0x0000	
Data	N/A	
Receive		Description
Length	size of (DeviceInfo)	DeviceInfo size
Data	DeviceInfo	DeviceInfo structure

```
1 DeviceInfo {
2     unsigned char hw_version;           //hardware version e.g. 'A' : A版
3     unsigned int fw_version;             //firmware version e.g. 0x00010200 : v1.2.0
4     unsigned int bl_version;             //bootloader version e.g. 0x00010200 : v1.2.0
5     unsigned char branch_code [8];      //Branch Code e.g. "40020000" : Ezcardio
6     Plus
7     unsigned char fs_version;            //file system version e.g. 0x01:v1
8     unsigned char reserved0[2];          //reserved 2 byte
9     unsigned short device_type;          //device type e.g. 0x8611: BP
10    unsigned short protocol_version;      //protocol version e.g.0x0100:v1.0
11    unsigned char cur_time[7];            //datetime e.g.0xE1070301090000:2017-03-01
12    09:00:00
13    unsigned short protocol_data_max_len; //max protocol data length
14    unsigned char reserved1[4];           //reserved 4 byte
15    SN{
16        unsigned char len;                //SN length, less than 18 e.g. 10
17        unsigned char serial_num[18];     //SN e.g. "2017022211" : 2017022211
18    }
19    unsigned char reserved2[4];           //reserved 4 byte
20 }
```

3.1.3 0xE2: Reset

Send		Description
Length	0x0000	
Data	N/A	
Receive		Description
Length	0x0000	
Data	N/A	

3.1.4 0xE3: Factory Reset

Send		Description
Length	0x0000	
Data	N/A	
Receive		Description
Length	0x0000	
Data	N/A	

3.1.5 0xE4: Battery

Send		Description
Length	0x0000	
Data	N/A	
Receive		Description
Length	size of (BatteryInfo)	BatteryInfo size
Data	BatteryInfo	BatteryInfo structure

```
1 BatteryInfo {
2   unsigned char state;      //battery state 0:in use 1:charging 2:charged 3:battery
   low
3   unsigned char percent;    //percent
4   unsigned short voltage;    //voltage e.g.  3950 : 3.95v
5 }
```

3.1.6 0xF1: Get file list

Send		Description
Length	0x0000	
Data	N/A	
Receive		Description
Length	size of (FileList)	FileList size
Data	FileList	FileList structure

```
1 FileList {
2     unsigned char file_num;        // file number
3     unsigned char *file_name[16]; // file name array, every 16 byte
4 }
```

3.1.7 0xF2: Read file start

Send		Description
Length	size of (FileReadStart)	FileStartRead size
Data	FileReadStart	FileStartRead structure
Receive		Description
Length	size of (FileStartReadReturn)	
Data	FileStartReadReturn	FileReadStart structure

```
1 FileReadStart {
2     unsigned char file_name[16];    //file name, 16 byte
3     unsigned int file_offset;        //offset to the file start
4     //unsigned char file_verson;     //file version, n not used
5 }
```

```
1 FileStartReadReturn {
2     unsigned int file_size;          //file size
3 }
```

3.1.8 0xF3: Read file content

Send		Description
Length	size of (FileRead)	FileRead
Data	FileRead	FileRead
Receive		Description
Length	size of (FileData)	
Data	FileData	file content

```
1 FileRead {
2   Unsigned int addr_offset;    //offset to file_offset
3 }
```

```
1 FileData {
2   Unsigned char *file_data;    //file content, max is protocol_data_max_len
3 }
```

3.1.9 0xF4: Read file end

Send		Description
Length	0x0000	
Data	N/A	
Receive		Description
Length	0x0000	
Data	N/A	

3.1.10 0xEC: Set time

Send		Description
Length	Size of(Time)	
Data	Time	
Receive		Description
Length	0x0000	
Data	N/A	


```

1 Time {
2     Unsigned short year;
3     Unsigned char month;
4     Unsigned char day;
5     Unsigned char hour;
6     Unsigned char minute;
7     Unsigned char second;
8 }

```

3.2 Private command(0x00 - 0xDF)

3.2.1 Blood Pressure Monitor (BP2/BP2A)

Bluetooth service & characteristic UUIDs

Service UUID : 14839ac47d7e415c9a42167340cf2339

Write Characteristic UUID: 8b00ace7eb0b49b0bbe99aee0a26e1a3

Notify Characteristic UUID: 0734594aa8e74b1aa6b1cd5243059a57

3.2.1.3 0x08: Get real-time data

Send		Description
Length	0x0000	
Data	N/A	
Receive		Description
Length	size of (RealTimeData)	RealTimeData size
Data	RealTimeData	RealTimeData structure

```

1 RunStatus {
2     unsigned char    status;        //state enum
3     BatteryInfo      battery;       //battery info
4     unsigned char    reserved[4];   //reserve 4
5 }

```

```

1 BatteryInfo {
2     Unsigned char state;        //battery state 0:in use 1:charging 2:charged 3:battery
    low
3     Unsigned char percent;      //percent
4     Unsigned short voltage;     //voltage e.g. 3950 : 3.95v
5 }

```

```

1  typedef enum {
2      STATUS_SLEEP = 0,           //sleep
3      STATUS_MEMORY,             //browser records
4      STATUS_CHARGE,             //charging
5      STATUS_READY,              //power on, ready
6      STATUS_BP_MEASURING,        //BP measuring
7      STATUS_BP_MEASURE_END,      //BP measure finished
8      STATUS_ECG_MEASURING,       //ECG measuring
9      STATUS_ECG_MEASURE_END,     //ECG measure finished
10 }status

```

Waveform sample value transform to actual voltage value:

$$x(n) = 1.0035 \cdot (n \cdot 1800) / (4096 \cdot 178.74)$$

```

1  waveform {
2      unsigned short sampling_num;           //wave_data length
3      short wave_data[sampling_num];        //wave_data
4  }

```

```

1  RealTimeWaveform{
2      unsigned char    type;           //Data type
3      Data             data;           //real-time data
4      waveform          wav;           //real-time waveform
5  }

```

data structure varies from type

```

1  // Type = 0 (BP measuring)
2  Data {
3      unsigned char    is_deflating;    //is deflating 0: false; 1: true
4      short            pressure;        //real-time pressure
5      unsigned char    is_get_pulse;    //pulse detected 0: false; 1: true
6      unsigned short    pulse_rate;     //pulse rate
7      unsigned char    reverse[14];     //reserved 14 byte
8  }
9
10 // Type = 1 (BP measure finsihed)
11 Data {
12     unsigned char    is_deflating;    //is deflating 0: false; 1: true
13     short            pressure;
14     unsigned short    systolic_pressure;
15     unsigned short    diastolic_pressure;
16     unsigned short    mean_pressure;
17     unsigned short    pulse_rate;
18     unsigned char    state_code;       //BP state code, see below table
19     unsigned char    medical_result;   //diagnostic, bit0:Irregular heart rate

```

```

20     unsigned char    reverse[7];        //reserved 7 byte
21 }
22
23 // Type = 2: (ECG measuring)
24 Data {
25     unsigned int      duration;          //measure duration
26     unsigned int      special_status;     //bit0: is weak signal, bit1:is lead off
27     unsigned short    pulse_rate;
28     unsigned char     reverse[10];       //reserved 10 byte
29 }
30
31 // Type = 3: (ECG measure finished)
32 Data {
33     unsigned int      result;             //ECG diagnostic result, see below table
34     unsigned short    hr;                 //HR: bpm
35     unsigned short    qrs;               //QRS: ms
36     unsigned short    pvcs;              //PVC个数
37     unsigned short    qtc;               //QTc: ms
38     unsigned char     reverse[8];        //reserved 8 byte
39 }

```

```

1 RealTimeData{
2     RunStatus        run_status;
3     RealTimeWaveform rt_wav;
4 }

```

BP state code

value	situation	Description
0x00	regular	regular
0x01	the sleeve is loose	unable analysis
0x02	disturb detected	
0x03		weak signal, no pulse detected
>=0x04	other error	device error

ECG diagnostic result

value	situation	Description
0x00000000	regular	regular ECG
0xFFFFFFFF	poor waveform quality	unable analysis
0xFFFFFFF0	lead off	unable analysis
0x00000001	HR>100bpm	high HR
0x00000002	HR<50bpm	low HR
0x00000004	R-R irregular	irregular heart beat
0x00000008	PVC	PVC
0x00000010	Cardiac arrest	Cardiac arrest
0x00000020	Atrial Fibrillation	Atrial Fibrillation
0x00000040	QRS>120ms	QRS wide
0x00000080	QTc>450ms	QTc long
0x00000100	QTc<300ms	QTc short

3.2.1.4 0x09: Switch device status

control the device status, like start a BP measure, send `Target_status = 0x00`

Send		Description
Length	0x0001	
Data	Target_status	
Receive		Description
Length	0x0000	
Data	N/A	

```

1 | typedef enum {
2 |     0,    //start measure BP
3 |     1,    //start measure ECG
4 |     2,    //browser records
5 |     3,    //power on, ready
6 |     4,    //power off
7 | }Target_status

```

3.2.2 ECG recorder (ER2)

Bluetooth service & characteristic UUIDs

Service UUID : 14839ac47d7e415c9a42167340cf2339

Write Characteristic UUID: 8b00ace7eb0b49b0bbe99aee0a26e1a3

Notify Characteristic UUID: 0734594aa8e74b1aa6b1cd5243059a57

3.2.2.1 0x01: Get real-time waveform

Send		Description
Length	0x0000	
Data	N/A	
Receive		Description
Length	size of (RealTimeWaveform)	RealTimeWaveform size
Data	RealTimeWaveform	RealTimeWaveform structure

Waveform sample value transform to actual voltage value:

$$x(n) = 1.0035 * (n * 1800) / (4096 * 178.74)$$

```
1 RealTimeWaveform {
2     unsigned short sampling_num;           //wave_data length
3     short         wave_data[sampling_num]; //wave_data
4 }
```

3.2.2.2 0x02: Get device run state

Send		Description
Length	0x0000	
Data	N/A	
Receive		Description
Length	size of (RunState)	RunState size
Data	RunState	RunState structure

```

1 RunState {
2     Unsigned short hr;           //hr, bpm
3     Unsigned char  sys_flag;
4     Unsigned char percent;       //battery percent e.g. 100:100%
5     Unsigned int  record_time;   //record time
6     Unsigned char  run_status;
7     Unsigned char  reserved[11]; //reserved 11 byte
8 }

```

sys_flag

bit	Description
0	R wave mark
1	if weak or no signal
2	if poor signal
3-5	reserved
6-7	battery state (0:in use 1:charging 2:charged 3:low battery)

run_status

bit	value	status
0-3	0x0	standby
	0x1	read to measure
	0x2	measuring
	0x3	analysis and saving
	0x4	saved
	0x5	measure less than 30s
	0x6	retry measure more than 6 times, go to standby
	0x7	lead off
4-7		last status, same to bit 0-3

3.2.2.3 0x03: Get real-time data

<i>Send</i>		Description
Length	0x0000	
Data	N/A	
<i>Receive</i>		Description
Length	size of (RealTimeData)	RealTimeData size
Data	RealTimeData	RealTimeData structure

```

1 RealTimeData{
2     RunState      run_state;
3     RealTimeWaveform rt_wav;
4 }

```

3.2.3 Visualbeat(ER1)

Bluetooth service & characteristic UUIDs

Service UUID : 14839ac47d7e415c9a42167340cf2339

Write Characteristic UUID: 8b00ace7eb0b49b0bbe99aee0a26e1a3

Notify Characteristic UUID: 0734594aa8e74b1aa6b1cd5243059a57

3.2.3.1 0x01: Get real-time waveform

<i>Send</i>		Description
Length	0x0000	
Data	N/A	
<i>Receive</i>		Description
Length	size of (RealTimeWaveform)	RealTimeWaveform size
Data	RealTimeWaveform	RealTimeWaveform structure

Waveform sample value transform to actual voltage value:

$$x(n) = 1.0035 * (n * 1800) / (4096 * 178.74)$$

```

1 RealTimeWaveform {
2     unsigned short sampling_num;           //wave_data length
3     short          wave_data[sampling_num]; //wave_data
4 }

```

3.2.3.2 0x02: Get device run state

Send		Description
Length	0x0000	
Data	N/A	
Receive		Description
Length	size of (RunState)	RunState size
Data	RunState	RunState structure

```
1 RunState {
2     Unsigned short hr;           //hr, bpm
3     Unsigned char  sys_flag;
4     Unsigned char percent;       //battery percent e.g. 100:100%
5     Unsigned int  record_time;   //record time
6     Unsigned char  run_status;
7     Unsigned char  reserved[11]; //reserved 11 byte
8 }
```

sys_flag

bit	Description
0	R wave mark
1	if weak or no signal
2	if poor signal
3-5	reserved
6-7	battery state (0:in use 1:charging 2:charged 3:low battery)

run_status

bit	value	status
0-3	0x0	standby
	0x1	read to measure
	0x2	measuring
	0x3	analysis and saving
	0x4	saved
	0x5	measure less than 30s
	0x6	retry measure more than 6 times, go to standby
	0x7	lead off
4-7		last status, same to bit 0-3

3.2.3.3 0x03: Get real-time data

Send		Description
Length	0x0000	
Data	N/A	
Receive		Description
Length	size of (RealTimeData)	RealTimeData size
Data	RealTimeData	RealTimeData structure

```

1 RealTimeData{
2     RunState      run_state;
3     RealTimeWaveform  rt_wav;
4 }
```